

## MedConnect Hackathon — Complete team guide

 **Confidential document — do not commit to Git, do not publish publicly.**

*This file contains 12 active API keys that grant direct access to the ThakaaMed AI inference engine. Distribute each row of the "Your credentials" table only to the relevant team. Keep your master copy in a password manager or an encrypted channel.*

### 1. Welcome

Welcome to the MedConnect Hackathon! For the duration of this event, **12 teams** will have direct access to our dental radiograph analysis engine, powered by artificial intelligence. It's the same engine that powers our production product: it automatically identifies teeth, caries, implants, prostheses, bone pathologies and much more — and returns everything as structured JSON, with bounding-box coordinates so you can layer your own visualisations on top.

Your mission: **build something remarkable** with these superpowers in 48–72 hours.

The API you're about to use serves real dental clinics in several countries every day. You're working with a professional tool, not a toy — so think big.

### 2. What the API does

The ThakaaMed API takes a **dental radiograph** (JPEG/PNG/converted DICOM) as input and returns a complete analysis including:

| Radiograph type       | Accepted                                                                            | Available models                                                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Panoramic             |  | Full detection: teeth, caries, restorations, implants, prostheses, periapical pathologies, bone level, calculus, sinuses, condyles |
| Periapical            |  | High-resolution focal detection on 1–4 teeth                                                                                       |
| Bitewing              |  | Inter-proximal caries, restorations, bone level                                                                                    |
| Lateral cephalometric |  | Cephalometric landmarks, calibration, Frankfort plane                                                                              |

For each detection, the API returns:

- The **type of structure or pathology** detected (tooth by FDI numbering, caries type, restoration type, etc.)
- A **confidence score** (0–1)

- The **bounding-box coordinates** (enclosing rectangle)
- The **polygonal coordinates** (exact shape of the detection — useful for precise overlays)
- The **associated tooth** (for example "tooth 36, occlusal caries")

The engine runs **191 different AI models** per panoramic radiograph. It's serious.

### 3. Your credentials

Each team receives:

- An **API key** (**api\_key**) — 32 uppercase hexadecimal characters.
- A **facility code** (**facility\_code**) — required on every call.
- **1,000 tokens** (1 token = 1 radiograph analysis, regardless of type).
- A **30-day validity period** (keys expire automatically on **2026-06-08**).

### 4. Endpoint and API call

#### Main URL (recommended — version 2.3)

POST <https://aiv4.thakaamed.com/api/v2.3/en/analyze/radiography/>

#### The **/xx/** segment in the URL = response language

The **/en/** segment in the URL controls **only the language of the text labels** returned in the JSON response (**label\_name**, descriptions, error messages). **The AI detection is strictly identical across all languages** — same models, same coordinates, same confidence scores, only the text is translated.

| Code      | Language   | Example URL                                          |
|-----------|------------|------------------------------------------------------|
| <b>en</b> | English    | <a href="#">.../api/v2.3/en/analyze/radiography/</a> |
| <b>fr</b> | French     | <a href="#">.../api/v2.3/fr/analyze/radiography/</a> |
| <b>ar</b> | Arabic     | <a href="#">.../api/v2.3/ar/analyze/radiography/</a> |
| <b>tr</b> | Turkish    | <a href="#">.../api/v2.3/tr/analyze/radiography/</a> |
| <b>de</b> | German     | <a href="#">.../api/v2.3/de/analyze/radiography/</a> |
| <b>nl</b> | Dutch      | <a href="#">.../api/v2.3/nl/analyze/radiography/</a> |
| <b>pt</b> | Portuguese | <a href="#">.../api/v2.3/pt/analyze/radiography/</a> |
| <b>ru</b> | Russian    | <a href="#">.../api/v2.3/ru/analyze/radiography/</a> |
| <b>es</b> | Spanish    | <a href="#">.../api/v2.3/es/analyze/radiography/</a> |

|    |       |                                      |
|----|-------|--------------------------------------|
| uz | Uzbek | .../api/v2.3/uz/analyze/radiography/ |
|----|-------|--------------------------------------|

💡 **Multilingual use-case idea:** for a clinic or app serving several communities (for example in Morocco: French + Arabic + English), do a **single POST**, remember the **slug**, then do **multiple GETs** on different language URLs — you'll get the same result, translated. Perfect for Challenge 1 (Patient Explainer) or Challenge 7 (Teleconsultation).

```
# Submit once (consumes 1 token)

slug = submit_image('panoramic.jpg') # POST on /en/


# Retrieve in 3 languages – free, same result translated

res_fr = requests.get('https://aiv4.thakaamed.com/api/v2.3/fr/analyze/radiography/',
params={'id': slug}).json()

res_ar = requests.get('https://aiv4.thakaamed.com/api/v2.3/ar/analyze/radiography/',
params={'id': slug}).json()

res_en = requests.get('https://aiv4.thakaamed.com/api/v2.3/en/analyze/radiography/',
params={'id': slug}).json()
```

### Async pattern: POST then GET

Analysing a radiograph takes between 5 and 30 seconds depending on image type and server load. The API therefore uses an **asynchronous two-step pattern**:

#### Step 1 — Submit the image (POST)

```
curl -X POST 'https://aiv4.thakaamed.com/api/v2.3/en/analyze/radiography/' \

-F 'api_key=YOUR_API_KEY' \

-F 'facility_code=YOUR_FACILITY_CODE' \

-F 'image=@path/to/radiograph.jpg'
```

Immediate response (200 OK):

```
{
  "status": true,
  "id": "9941c5cf-6e4f-4f02-9425-...",
  "results": "https://aiv4.thakaamed.com/api/v2.3/en/analyze/radiography/?id=9941c5cf-...",
  "Warning__Message!!!": "No Warning."
}
```

- **status: true** here means "**submission accepted**" — *not* that the analysis is done.
- **id** is your **analysis slug**. Keep it for polling.
- **results** here is the **ready-made GET URL** (convenient).
- **Warning\_\_Message!!!** (sic) flags issues like undersized images — analysis still proceeds.

### Step 2 — Retrieve the result (GET)

```
curl 'https://aiv4.thakaamed.com/api/v2.3/en/analyze/radiography/?id=9941c5cf-...'
```

Analysis usually completes in **5 to 30 seconds** (often <10 s on the hackathon infrastructure). The completion signal is **is\_done: true** in the GET response — **NOT the `status` field**, which only appears in the POST response.

```
{
  "id": "...",
  "is_done": true,           // ← your completion signal
  "error_status": false,
  "message": "Analysis completed successfully.",
  "results": { /* ... see §5 ... */ }
}
```

Set up a polling loop with a **2 to 5 second** delay between requests.

### Python example (minimal client)

```
import requests
import time

API_KEY = 'YOUR_API_KEY'
FACILITY = 'YOUR_FACILITY_CODE'
BASE = 'https://aiv4.thakaamed.com/api/v2.3/en/analyze/radiography/'

def analyze(image_path):
    # Submit
    with open(image_path, 'rb') as f:
        r = requests.post(BASE, data={
            'api_key': API_KEY,
            'facility_code': FACILITY,
```

```

        }, files={'image': f})

    r.raise_for_status()

    slug = r.json()['id']

    # Poll – wait for `is_done: true`
    for _ in range(60): # max 60 attempts ≈ 3 minutes
        time.sleep(3)

        r = requests.get(BASE, params={'id': slug})

        data = r.json()

        if data.get('is_done') is True:
            if data.get('error_status'):
                raise RuntimeError(data.get('error_message') or data.get('message'))

            return data

    raise TimeoutError(f"Analysis {slug} not complete after 3 minutes")

result = analyze('panoramic_test.jpg')
print(result)

```

### JavaScript example (Node.js / browser)

```

const API_KEY = 'YOUR_API_KEY';

const FACILITY = 'YOUR_FACILITY_CODE';

const BASE = 'https://aiv4.thakaamed.com/api/v2.3/en/analyze/radiography/';

async function analyze(imageFile) {
    const form = new FormData();

    form.append('api_key', API_KEY);

    form.append('facility_code', FACILITY);

    form.append('image', imageFile);

    const submit = await fetch(BASE, { method: 'POST', body: form });

    const { id: slug } = await submit.json();
}

```

```

for (let i = 0; i < 60; i++) {

  await new Promise(r => setTimeout(r, 3000));

  const poll = await fetch(`${BASE}?id=${slug}`);

  const data = await poll.json();

  if (data.is_done === true) {

    if (data.error_status) throw new Error(data.error_message || data.message);

    return data;

  }

}

throw new Error(`Timeout for ${slug}`);
}

```

## 5. Anatomy of a response

When your GET succeeds (`is_done: true`), you receive a deep, richly structured JSON (~300 KB to 1 MB depending on the number of detections). **Top-level** fields:

| Field                             | Description                                                                                                                                                                                                      |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>                   | The slug you received on POST.                                                                                                                                                                                   |
| <code>is_done</code>              | <code>true</code> when analysis is complete. <b>This is your completion signal.</b>                                                                                                                              |
| <code>error_status</code>         | <code>true</code> on a server-side error (unreadable image, etc.).                                                                                                                                               |
| <code>message</code>              | Human-readable status (e.g. <code>"Analysis completed successfully."</code> ).                                                                                                                                   |
| <code>error_message</code>        | Details if <code>error_status</code> is true.                                                                                                                                                                    |
| <code>version</code>              | Engine version that produced the result.                                                                                                                                                                         |
| <code>response_time</code>        | Server-side compute time (seconds).                                                                                                                                                                              |
| <code>original_image</code>       | URL of the image you uploaded (ThakaaMed CDN).                                                                                                                                                                   |
| <code>draw_image</code>           |  <b>URL of an annotated PNG</b> (overlays + bounding boxes drawn) — ready to display as-is.                                   |
| <code>embedded_link</code>        |  <b>URL of an embeddable web viewer</b> (drop in an <code>&lt;iframe&gt;</code> ) showing the image + interactive detections. |
| <code>embedded_report_link</code> | URL of a printable PDF report.                                                                                                                                                                                   |
| <code>results</code>              | <b>Object</b> (not array!) containing the structured analysis. Detailed below.                                                                                                                                   |

 **Hackathon tip:** `draw_image` and `embeded_Link` are superpowers. You can ship an impressive visual demo just by displaying `draw_image` (overlays already drawn for you) or by embedding `embeded_Link` in an `<iframe>` — no need to write your own canvas renderer.

### Structure of results

```
results: {  
  
  implant_brands: [],  
  
  tooth_results: {  
  
    "11": {                                     // ← key = FDI tooth numbering (11-48)  
  
      slug: "...",  
  
      label_slug: "...",  
  
      is_missing: false,  
  
      illnesses: [                             // ← pathologies detected on this tooth  
  
        {  
  
          name: "Dental Pulp",  
  
          slug: "...",  
  
          probability: 42.39,                 // ← scale 0-100, NOT 0-1  
  
          icd_dict: { icd_code: "Z97.2", icd_desc: "Filling-Overhanging margin" }  
  
        }  
  
      ],  
  
      treatment_methods: [],  
  
      coordinates: {                          // ← bounding box  
  
        xmin: 1128.0, ymin: 526.0,  
  
        xmax: 1214.0, ymax: 768.0,  
  
        proba: 88.81                         // ← tooth confidence (0-100)  
  
      },  
  
      list_coordinates: [[x,y], [x,y], ...] // exact polygon  
  
    },  
  
    "12": { ... },  
  
    ...  
  
  }  
  
}
```

### Important:

- `coordinates` uses keys `xmin/ymin/xmax/ymax` (not `x1/y1/x2/y2`), with a `proba` field on a 0–100 scale.
- `illnesses[].probability` is also on a 0–100 scale.
- `tooth_results` is a **dictionary** keyed by FDI number ("`11`", "`12`", ..., "`48`"), not a list.
- A typical panoramic returns ~32 teeth with ~30–50 total pathologies.

 **Tip:** a real, complete sample is at `examples/sample_v23_analysis.json` (~1.3 MB). Open it in VSCode or explore with `jq` to map the structure before burning tokens.

## 6. Quotas, limits, and best practices

| Resource            | Per-team limit                                                                |
|---------------------|-------------------------------------------------------------------------------|
| Tokens              | 1,000 analyses (1 token consumed per analysis, regardless of image type)      |
| Lifetime            | 30 days (expires 2026-06-08)                                                  |
| Max image size      | ~50 MB per file (recommended: <10 MB, JPEG/PNG)                               |
| Concurrent requests | No hard limit, but be reasonable — infrastructure is shared between all teams |
| Image format        | JPEG, PNG. DICOM not supported directly — convert client-side                 |

### If you run out of tokens

Contact the organisers. A **+500 token** top-up may be granted mid-event for a justified need (for example, you're building a batch pipeline and need to test on 200 images).

### Confidentiality — IMPORTANT

 **Do not send any real patient data without explicit consent.** For the hackathon, use:

- **Public datasets** of dental radiographs (e.g. on Kaggle, GitHub, Hugging Face);
- **Synthetic** or **anonymised** images that the organisers will provide;
- Your own radiographs if a team member is a dentist and has access to their own images.

The API is **not a certified medical device** in this context. Your prototypes are hackathon demos, not clinical tools. State this clearly in any user interface you build.

## 7. Challenge ideas

Here are **12 project ideas** you can pick up (or that can inspire your own). Pick the one that resonates with your team — or combine several. Each idea is designed to be achievable in 48–72 hours by a team of 3–5 people.



### Challenge 1 — The Patient Explainer

*"Mrs Smith, here is your X-ray. The AI found 3 cavities and a healthy implant. Would you like me to explain each one?"*

Build a web or mobile app that:

- Takes a radiograph as input;
- Displays the result in **accessible language** (no dental jargon);
- Generates an **audio explanation** (text-to-speech) for patients who prefer listening;
- Offers a **multilingual version** (French, Arabic, English — useful in multicultural practices);
- Includes **simplified illustrations** (e.g. "this is what a cavity looks like, this is how it progresses if untreated").

**Bonus:** integrate an LLM (Claude, GPT, Mistral) that answers patient questions about their X-ray in real time ("and that spot there, what is it?").

### Challenge 2 — The AI Second Opinion

*"My dentist told me I need 3 crowns. I want a second opinion."*

The user uploads:

1. Their radiograph;
2. The **treatment plan proposed by their dentist** (free text).

Your app:

- Compares the AI detections with the proposed plan;
- Identifies **suggested treatments without a corresponding AI detection** (potential over-prescription);
- Identifies **AI detections not covered** by the plan (potential under-prescription);
- Presents a clear **divergence report**, with confidence scoring.

**Bonus:** hook in an LLM to generate a natural-language summary and questions to ask their dentist.

### Challenge 3 — The Emergency Triage Agent

*"47 patients in the waiting room. Who do we see first?"*

For overwhelmed clinics (NGOs, humanitarian missions, low-density countries):

- Simple camera interface → snap a photo of the X-ray on screen, or capture directly;
- AI analyses in real time;
- Computes an **urgency score** (e.g. active periapical abscess > deep caries > superficial caries > scaling);
- Automatically reorders the waiting list and alerts the dentist on critical cases.

**Bonus:** offline mode with cached analyses for low-connectivity zones.

### Challenge 4 — The Anti-Anxiety App for Children

*"Don't be scared, Mr Cavity is just a little monster — watch, the dentist is going to chase him away!"*

Turn the anxious X-ray experience into a game:

- The child uploads their X-ray (or their parents do);
- The app **transforms the black-and-white X-ray into a colourful, friendly illustration**;
- Each cavity becomes an **animated character** ("Mr Cavity", "Princess Filling", etc.);
- An interactive story narrates what's going to happen at the dentist;
- **Reward** system for post-visit brushing.

**Bonus:** image generation via Stable Diffusion or DALL-E from the detection coordinates.

### **Challenge 5 — The Dental Student Companion**

*"I'm in my 4th year. I want to practise reading X-rays."*

Gamified learning platform:

- Library of radiographs (public datasets);
- The student **annotates the X-ray** (clicks on caries, restorations, etc.);
- AI compares their annotations to those of the ThakaaMed model;
- **Score** per exercise, **leaderboard** between students, **statistics** by pathology type;
- "Clinical cases" mode: 10 X-rays per session, detailed debrief at the end.

**Bonus:** build an LLM assistant that acts as a tutor ("why did you miss that cavity? look at zone X...").

### **Challenge 6 — The Oral Health Logbook**

*"I have my X-rays from the past 5 years. Is my condition getting worse?"*

Personal application (web or mobile):

- The user uploads several X-rays taken at different dates;
- AI analyses each;
- The app **detects evolution**: new caries, successful treatment, bone loss, etc.;
- **Timeline** visualisation with oral health curves;
- Personalised recommendations ("you have 3 new cavities in 1 year — increase your visits to 2x/year").

**Bonus:** prediction via simple ML model — "if you don't treat, here's where you'll be in 2 years".

### **Challenge 7 — The Teleconsultation Assistant**

*"The nearest village with a dentist is 200 km away."*

For medical deserts:

- A nurse or general practitioner takes the X-ray in a local clinic;
- AI does an **automatic pre-analysis**;
- A **pre-filled report** is generated and sent to a remote specialist dentist;
- The specialist validates / modifies / comments;
- The final report is returned to the patient within 24 h.

**Bonus:** smart queue — the most urgent cases (per the AI) rise to the top of the specialist's list.

### Challenge 8 — The Cost & Pre-Authorisation Estimator

*"How much will this proposed treatment cost me?"*

Financial transparency app:

- Upload X-ray + city/country of residence;
- AI detects probable required treatments;
- The app cross-references with a **pricing database** (public or one you build by hand);
- Generates a **price range** per treatment ("molar filling: 80–150 € depending on clinic");
- Generates a **pre-formatted insurance/mutual letter** with corresponding CCAM/ICD-10 codes.

**Bonus:** integration with existing dental price-comparator APIs.

### Challenge 9 — The Smart Dental Chatbot (RAG over X-rays)

*"Here's my X-ray. Tell me everything."*

Combine the ThakaaMed engine with a modern LLM:

- The user uploads their X-ray;
- ThakaaMed AI produces the JSON analysis;
- You **inject this JSON into the context of an LLM** (Claude, GPT, Mistral);
- The user can **chat**: "explain this detection to me", "is it serious?", "how long before it gets worse?", "what questions should I ask my dentist?".

**Bonus:** add a "cite your sources" mode where the LLM ties each statement to a precise detector in the JSON ("per the level-2 occlusal caries detector, tooth 16, confidence 0.91...").

### Challenge 10 — 3D / Augmented Reality Visualisation

*"Wow, this is so cool."*

Build something visually spectacular:

- From the API's polygonal coordinates, generate an **approximate 3D model** of the teeth;
- AR mode (WebXR or mobile app): the phone camera shows detections **overlaid on the patient's face** in real time;
- Animations: show the cavity "drilling" into the tooth, the implant integrating into bone, etc.

**Bonus:** VR mode for immersive headsets — the user walks around inside their own mouth.

### Challenge 11 — Epidemiological Mapping

*"What percentage of the population has untreated caries in this region?"*

Public health tool:

- Import anonymised public X-ray datasets by geographic zone;
- Run the AI over a large number of images (batch pipeline — use your 1,000 tokens strategically!);
- Build an **interactive map** of dental pathologies by region;
- Identify **patterns**: zones with high caries incidence? implants? bone loss?

**Bonus:** export your results as a report for NGOs or public-health ministries.

## Challenge 12 — Surprise us!

You have an idea no one has thought of? Go for it. Some very loose pointers:

- 🎮 **Video game** where the hero cleans giant teeth (cavities are enemies);
- 🎵 **Music generation**: each tooth is a note, each cavity is a dissonance — play your own X-ray as a symphony;
- 🐕 **Cross-species detection**: test whether our models work (or not) on canine/feline X-rays, and publish the report;
- 📄 **Auto-generation of legal reports** for insurance, with professional PDF formatting;
- 🔔 **Family alert**: "dad, your tooth 26 has gotten worse since last year, book an appointment" — smart push notifications;
- 🦷 **Dental R&D tool**: aggregate statistics over hundreds of X-rays to test a clinical hypothesis.

**The golden rule: have fun, be ambitious, and build something you'll be proud of.**

---

## 8. Practical tips to get started

### The first 30 minutes

3. **Make a test call immediately** with a sample X-ray. Save the JSON response to a file. Explore its structure in a JSON editor (VSCode, jq, Postman). The bundle ships with `data/samples/panoramic/` and `data/samples/bitewing/` images — use them.
4. **Validate your `facility\_code`** — without it, the v2.3 API returns a 400 error. It's the #1 cause of being stuck at the start of a hackathon.
5. **Set up polling early** — the classic mistake is a too-early GET and concluding the API is broken. Wait 3–5 seconds minimum.
6. **Cache your results** — every call costs 1 token. In dev, save the JSON locally and work on it offline so you don't burn tokens.

### Common errors

| Error                                   | Likely cause                                | Fix                                      |
|-----------------------------------------|---------------------------------------------|------------------------------------------|
| 400 Bad Request – Facility key required | <code>facility_code</code> missing or empty | Add the field to your form-data          |
| 404 / wrong API key                     | Typo in <code>api_key</code>                | Exact copy-paste, watch for spaces       |
| Insufficient tokens                     | You've used your 1,000 tokens               | Contact organisers for a top-up          |
| Image too large                         | File > 50 MB                                | Compress or resize client-side           |
| Empty response after 1 minute           | Corrupt or unsupported format               | Check the file in a viewer before upload |

### Managing your 1,000-token quota

- **Phase 1 (exploration, day 1)**: ~50 calls max. Learn the response structure, test 3–4 different X-ray types.
- **Phase 2 (development, days 1–2)**: ~200 calls. Cache aggressively. Reuse saved JSON.
- **Phase 3 (final demo and video)**: keep ~100–200 tokens for nice demos on real images.
- **Reserve**: ~500 tokens for the unexpected. Trust us, you'll need them.

### Recommended tools

- **Postman** or **Insomnia** to explore the API manually before coding.
- **jq** at the command line to navigate the JSON response.
- **Streamlit** or **Gradio** (Python) for ultra-fast UI prototypes.
- **Next.js** or **Vite + React** for something more polished.
- **Claude API** or **OpenAI API** for the LLM/conversational layer (if your challenges call for it).

## 9. Radiograph datasets (to test without real patients)

We ship **154 anonymised dental radiographs** in this bundle:

- `data/samples/panoramic/` — 41 panoramic X-rays
- `data/samples/bitewing/` — 113 intra-oral X-rays (RVG, periapical)

All metadata has been stripped (no EXIF, no JPEG comments, no patient identifiers in filenames). Use these freely.

Need more variety? A few public sources:

| Source                                  | Type      | Link                                           |
|-----------------------------------------|-----------|------------------------------------------------|
| <b>Kaggle — Dental Panoramic X-rays</b> | Panoramic | Search "dental panoramic" on kaggle.com        |
| <b>Hugging Face — Dental Datasets</b>   | Mixed     | huggingface.co/datasets (search "dental")      |
| <b>GitHub — open-dental-datasets</b>    | Mixed     | GitHub search for "dental radiography dataset" |
| <b>Synthetic data</b>                   | All types | Ask the organisers                             |

*If you have a dentist or dental student on your team, they can often supply training images from their school — just verify it's anonymised.*

## 10. Evaluation criteria (suggested)

The jury will likely score your project on:

| Criterion                                                                    | Suggested weight |
|------------------------------------------------------------------------------|------------------|
| <b>Real impact — does your project solve a real dental problem?</b>          | 30%              |
| <b>Technical quality — clean code, solid API integration, error handling</b> | 25%              |
| <b>Creativity and originality — did you do something surprising?</b>         | 20%              |
| <b>Polish and UX — is the user experience pleasant?</b>                      | 15%              |
| <b>Presentation and demo — can you tell your story?</b>                      | 10%              |

***Bonus tip:** record a **demo video** of at least 2 minutes. Many strong projects get edged out by weak presentations.*

---

## 11. Frequently asked questions

**Q: Does the API work on animal dental X-rays (dogs, cats)?** A: The models were trained exclusively on human X-rays. You can test out of curiosity, but results won't be reliable. (That said, it's an interesting challenge in itself — see Challenge 12.)

**Q: Can I use an LLM (Claude, GPT) on top of the ThakaaMed API?** A: Absolutely yes — it's even strongly encouraged. Your ThakaaMed keys only cover ThakaaMed; LLMs require your own keys (free accounts available on Anthropic, OpenAI or Mistral).

**Q: Are my 1,000 tokens shared between team members?** A: Yes. All calls with your `api_key` draw from the same 1,000-token pool.

**Q: Can I save the JSON responses and work offline?** A: Yes, that's actually the best practice for saving tokens. Download 20–30 reference JSONs at the start of the hackathon and work with them in dev.

**Q: What happens after 2026-06-08?** A: Your key becomes inactive. The API will return a quota error. If you want to continue your project after the hackathon, contact ThakaaMed for a commercial licence.

**Q: Does the API return ICD-10 or CCAM codes?** A: ICD-10 mapping exists but isn't enabled by default on your keys (to keep things simple). If your project needs it (e.g. Challenge 8), ask the organisers to enable the `icd_status` flag on your account — it's free and quick.

**Q: The server feels slow / my calls are failing — what's going on?** A: The infrastructure is shared across all 12 teams. If everyone runs batches at the same time, things slow down. Be considerate, spread out your calls. If you suspect an outage, contact the organisers.

---

## 12. Support and contact

- **Hackathon coordinator:** Rime AABIL - Tel: +212708150051
  - For technical issues with the API (unexpected responses, persistent errors, token top-ups), contact the hackathon coordinator first — they bridge to the ThakaaMed team.
- 

## 13. Medical and ethical disclaimer

 **Mandatory reading before the final demo.**

The ThakaaMed AI models used here are diagnostic-aid tools designed for qualified dentists in clinical practice. **No prototype built during this hackathon should be used directly on patients without validation by a qualified dentist.**

If your user interface targets the general public (patients), **always** include:

- A clear notice: "⚠️ This tool is a demonstration prototype. It does not replace a consultation with a qualified dentist."
- No promise of a definitive diagnosis.
- No treatment prescription.

Respect confidentiality: do not store any patient image without written consent, and anonymise everything you present in public (demo video, screenshots, etc.).

---

## 14. Going further

Once the hackathon is over, if your project has potential:

- ThakaaMed offers **commercial licences** to integrate the engine into real products.
  - Several of the challenges above match **current business opportunities** in the digital dental health sector — feel free to turn your prototype into a startup.
  - Stay in touch with the ThakaaMed team via the hackathon coordinators.
- 

**Good luck to all teams!** 🦷 ⚡

*May the best project win — and above all, may you learn and have fun. That's priority number one.*